

C02 — Import i porządkowanie danych ankietowych

Zarządzanie informacją — część ilościowa — 2026/27

Marek Deja

2026/27

Spis treści

Cel i kolejność pracy	1
Start w RStudio	2
Co zawiera plik	2
CSV: separator, liczby i polskie znaki	2
Krótki most do arkusza — demonstracja	2
Skala pomiarowa i klasa R	3
Zmienne ilościowe	3
Zmienne nominalne	3
Zmienna porządkowa	3
Dwie szybkie kontrole	3
Jawne czyszczenie krok po kroku	4
Powtórzone wiersze	4
Kody braków i zakres	4
Uporządkowane etykiety	4
Dziennik i eksport	5
Z02 — własny wariant ankiety	5
Oddanie i pytania kontrolne	6
Cel i kolejność pracy	

Pytanie na dziś brzmi: czy tabela po imporcie rzeczywiście zawiera odpowiedzi, które miała zawierać? Zanim obliczymy średnią, sprawdzamy separator, kodowanie, jednostki, kody braków i powtórzone wiersze. Po ćwiczeniu przygotujesz odtwarzalny zapis czyszczenia własnego wariantu danych. Nie wykonujemy jeszcze testu hipotezy.

Minuty	Etap
0–10	Logowanie i odtworzenie projektu
10–30	Import CSV, typy i słownik
30–55	Wspólne czyszczenie i eksport
55–80	Z02 na własnych danych
80–90	Kontrola, wysyłka, wylogowanie

Start w RStudio

Uruchom pakiet i logowanie tak jak w C01. Wybierz własny pseudonim i adres repozytorium. Otwarcie głównego projektu ustala katalog, więc ścieżka `dane/surowe.csv` wskazuje plik Twojej pracy, a nie plik z poprzedniej grupy.

```
library(badaniaZI)
zaloguj_github()
rozpocznij_zajecia("C02", "s017",
  "https://github.com/prowadzacy/badania-s017.git")
```

Co zawiera plik

Wiersz tabeli ma oznaczać jednego `id_odpowiedzi`. Kolumna `id_odpowiedzi` jest pseudonimem obserwacji, `grupa` etykietą jednej z dwóch kategorii, `czestosc_korzystania` odpowiedzią porządkową 1–5, a `czas_wyszukiwania` czasem w minutach. Pozycje `pozycja_1`–`pozycja_6` mają odpowiedzi 1–5; kolumny zaczynające się od `kanal_` oznaczają zaznaczenie kanału 1 lub niezaznaczenie 0. Dane są syntetyczne: ćwiczymy analizę, nie opisujemy rzeczywistych użytkowników katalogu.

Słownik wyjaśnia znaczenie kolumn, a manifest określa wariant i wersję generatora. W tym zbiorze kod 99 w pozycji 2 oznacza brak odpowiedzi; nie jest wysoką oceną. W innym zbiorze 99 mogłoby mieć inne znaczenie, dlatego regułę odczytuje się z dokumentacji. NA jest brakiem znanej wartości, natomiast zero w kanale jest odpowiedzią negatywną.

CSV: separator, liczby i polskie znaki

CSV jest plikiem tekstowym. `read.csv()` zakłada przecinek między kolumnami i kropkę dziesiętną; `read.csv2()` zakłada średnik i przecinek dziesiętny. Nie wybieraj funkcji na podstawie systemu operacyjnego: obejrzyj początek pliku albo dokumentację eksportu. `encoding="UTF-8"` pozwala poprawnie odczytać etykiety z polskimi znakami.

Wspólnie odczytujemy przykład z pakietu, który jest dostępny bez internetu. Funkcja zwraca ścieżkę do pliku; to `read.csv()` tworzy tabelę. Odczytaj liczbę wierszy, kolumn i pierwsze nazwy grup, aby wykryć zły separator albo kodowanie.

```
surowe <- utils::read.csv(badaniaZI::plik_przykladu("csv"),
  encoding = "UTF-8", stringsAsFactors = FALSE)
dim(surowe)

## [1] 150 15

head(surowe[, c("id_odpowiedzi", "grupa", "czas_wyszukiwania", "pozycja_2")])
```

```
##   id_odpowiedzi      grupa czas_wyszukiwania pozycja_2
## 1      o001      nowi          12.1           3
## 2      o002      nowi          10.4           3
## 3      o003 doświadczeni          5.8           4
## 4      o004 doświadczeni          16.1          4
## 5      o005 doświadczeni          28.0           2
## 6      o006 doświadczeni           6.3           4
```

Jedna kolumna zamiast piętnastu zwykle wskazuje niewłaściwy separator. Tekst zamiast liczb może oznaczać niezgodny znak dziesiętny lub dopisek jednostki, np. „12 min”. Nie naprawiaj danych przez losowe próby konwersji całej tabeli: ustal, która kolumna jest błędna i dlaczego.

Krótki most do arkusza — demonstracja

XLSX może mieć wiele arkuszy, formuły, formatowanie i puste wiersze. Pakiet zawiera mały arkusz z pierwszymi dwunastoma odpowiedziami przykładu CSV, a nie drugi pełny zbiór. Tę różnicę trzeba zapisać przed porównaniem liczebności. Na ćwiczeniu obowiązkowy import dotyczy CSV; import arkusza służy rozpoznaniu formatu.

Poniższy kod wymaga opcjonalnego pakietu `readxl`, przygotowanego w sali. Wybieramy arkusz jawnie, żeby nie odczytać przypadkiem strony z opisem. Odczytaj strukturę i sprawdź, czy nagłówki stały się nazwami kolumn.

```
arkusz <- readxl::read_excel(badaniaZI::plik_przykladu("xlsx"), sheet = 1)
str(arkusz)
nrow(arkusz)
```

Skala pomiarowa i klasa R

Skala mówi, co znaczą wartości; klasa mówi, jak R je przechowuje. Identyfikator zapisany liczbą nie staje się pomiarem ilościowym. Nie liczymy średniej z pseudonimów ani grup. `factor` przechowuje etykiety kategorii, a `ordered factor` dodatkowo ich kolejność; techniczna zamiana nie nadaje jednak skali równych odległości.

Zmienne ilościowe

Zmienna ilościowa przechowuje liczby, które można sensownie porównywać i podsumowywać średnią, medianą lub odchyleniem standardowym. W ankiecie przykładem jest czas znalezienia informacji w minutach. Częstotliwość korzystania zakodowana od 1 do 5 pozostaje zmienną porządkową, a nie liczbą wizyt. Patrzymy na zakres wartości, bo wartości skrajne lub nierealne często wskazują problem w danych.

Zmienne nominalne

Grupa „studenci studiów licencjackich” albo „studenci studiów magisterskich” jest kategorią bez naturalnego porządku. Nominalna kolumna wymaga liczebności lub odsetków, a nie średniej jej kodów. Te same kody 1 i 2 mogą w innej ankiecie oznaczać inne grupy.

Zmienna porządkowa

Skala oceny dostępności informacji ma porządek od niskiej do wysokiej oceny, ale odległość między 2 i 3 nie musi oznaczać dokładnie tego samego co między 4 i 5. Dlatego w analizie opisowej często pokazujemy rozkład odpowiedzi, a nie tylko średnią. Patrzymy na poziomy skali i liczebność każdej oceny.

Dwie szybkie kontrole

`str()` jest jedną z najważniejszych funkcji po wczytaniu danych. Pokazuje liczbę obserwacji, nazwy kolumn i klasy zmiennych w zwartej formie. Patrzymy na to, czy zmienne katgoryczne są faktorem, a zmienne liczbowe są zapisane jako `num` albo `int`.

```
wybrane <- c("id_odpowiedzi", "grupa", "czas_wyszukiwania")
str(surowe[, c(wybrane, "czestosc_korzystania")])
```

```
## 'data.frame':    150 obs. of  4 variables:
## $ id_odpowiedzi      : chr  "o001" "o002" "o003" "o004" ...
## $ grupa              : chr  "nowi" "nowi" "doświadczeni" "doświadczeni" ...
## $ czas_wyszukiwania  : num  12.1 10.4 5.8 16.1 28 6.3 8.1 24.4 18.4 18.7 ...
## $ czestosc_korzystania: int   4 2 3 NA 2 5 5 4 4 4 ...
```

```
badaniaZI::tabela_klas_r(surowe[, wybrane])
```

```
##              zmienna   klasa_R typ_wewnetrzny
## id_odpowiedzi   id_odpowiedzi character      character
## grupa           grupa character      character
## czas_wyszukiwania czas_wyszukiwania  numeric      double
```

`summary()` dopasowuje opis do typu zmiennej: dla liczb pokazuje kwartyle, a dla faktorów liczebności poziomów. To szybki sposób wykrywania błędów, zanim zaczniemy wizualizację lub testy. Patrzymy na minimum, maksimum i najczęstsze kategorie.

```
summary(surowe[, c("czas_wyszukiwania", "pozycja_2", "czestosc_korzystania")])
```

```
##   czas_wyszukiwania   pozycja_2   czestosc_korzystania
```

```
## Min. : -5.00 Min. : 1.000 Min. :1.00
## 1st Qu.: 5.80 1st Qu.: 3.000 1st Qu.:2.00
## Median : 8.25 Median : 3.000 Median :3.00
## Mean : 16.78 Mean : 5.847 Mean :3.11
## 3rd Qu.: 13.00 3rd Qu.: 4.000 3rd Qu.:4.00
## Max. :999.00 Max. :99.000 Max. :5.00
## NA's :4 NA's :4
```

Minimum -5 minut i maksimum 999 minut trzeba porównać ze słownikiem. W tym przykładzie dopuszczalny zakres wynosi 0–120 minut. Nietypowa, lecz dopuszczalna wartość nie jest automatycznie błędem; zakres określono przed analizą. Całego `id_odpowiedzi` nie usuwa się dlatego, że jedno jego pole wymaga oznaczenia braku.

Jawne czyszczenie krok po kroku

Powtórzone wiersze

Powtórzenie identyfikatora i identyczność całego wiersza to dwie różne kontrole. Generator celowo dopisał dwie identyczne kopie odpowiedzi. Zachowujemy pierwszą kopię każdego całego wiersza; gdyby to samo ID miało różne odpowiedzi, należałoby wyjaśnić konflikt, zamiast arbitralnie wybrać ostatnią.

Tworzymy nową tabelę, pozostawiając surowe bez zmian. `duplicated()` wskazuje kolejne identyczne wiersze, a `!` odwraca ten warunek. Odczytaj `N` przed i po oraz sprawdź, czy identyfikatory pozostały unikalne.

```
n_przed <- nrow(surowe)
liczba_duplikatow <- sum(duplicated(surowe))
dane <- surowe[!duplicated(surowe), ]
c(przed = n_przed, po = nrow(dane), kopie = liczba_duplikatow)
```

```
## przed po kopie
## 150 148 2
```

```
anyDuplicated(dane$id_odpowiedzi)
```

```
## [1] 0
```

Kody braków i zakres

Rekodujemy tylko kod opisany w słowniku. Znak `==` porównuje wartości, a warunek `!is.na()` chroni przed nieznany wynik porównania. Zapis `dane$pozycja_2[kod_99] <- NA` zmienia wybrane komórki; nie usuwa wierszy.

```
kod_99 <- !is.na(dane$pozycja_2) & dane$pozycja_2 == 99
liczba_99 <- sum(kod_99)
dane$pozycja_2[kod_99] <- NA
poza_zakresem <- !is.na(dane$czas_wyszukiwania) &
  (dane$czas_wyszukiwania < 0 | dane$czas_wyszukiwania > 120)
liczba_czasow <- sum(poza_zakresem)
dane$czas_wyszukiwania[poza_zakresem] <- NA
c(kod_99 = liczba_99, czasy_poza_zakresem = liczba_czasow)
```

```
## kod_99 czasy_poza_zakresem
## 4 2
```

Uporządkowane etykiety

Pojedynczą pozycję 1–5 pokazujemy z zachowaniem kolejności. `levels=1:5` określa kody, a `ordered=TRUE` przechowuje ich porządek. Liczebności obejmują także brak; nie włączaj 99 jako szóstego poziomu oceny.

```
dane$grupa <- factor(dane$grupa)
ocena_opis <- factor(dane$pozycja_1, levels = 1:5,
  labels = c("1 zdecydowanie nie", "2 raczej nie", "3 ani tak, ani nie",
    "4 raczej tak", "5 zdecydowanie tak"),
  ordered = TRUE)
table(ocena_opis, useNA = "ifany")

## ocena_opis
## 1 zdecydowanie nie      2 raczej nie 3 ani tak, ani nie      4 raczej tak
##                1                24                54                45
## 5 zdecydowanie tak      <NA>
##                20                4
```

Nie stosuj `as.numeric()` do faktora, żeby odzyskać pierwotne liczby: otrzymasz numery poziomów. Najpierw obejrzyj `levels()`, a ewentualną konwersję uzasadnij. W analizie zachowujemy surowe numery pozycji w osobnych kolumnach, aby później jawnie odwrócić pozycję 3 i utworzyć indeks.

Dziennik i eksport

Dziennik łączy regułę z jednostką liczenia zmiany. Usunięte duplikaty liczymy w wierszach, natomiast rekodowanie pozycji i czasu w komórkach. Zapisujemy osobno N przed i po, żeby nie sumować nieporównywalnych wielkości.

```
dziennik <- data.frame(
  regula = c("identyczne kopie", "pozycja_2: 99 na NA", "czas poza 0-120 na NA"),
  jednostka = c("wiersze", "komórki", "komórki"),
  liczba = c(liczba_duplikatow, liczba_99, liczba_czasow))
dziennik

##                regula jednostka liczba
## 1      identyczne kopie   wiersze      2
## 2   pozycja_2: 99 na NA   komórki      4
## 3   czas poza 0-120 na NA   komórki      2
```

Eksport zapisuje wynik pracy, a nie zmienia źródło. `row.names=FALSE` zapobiega dopisaniu złej kolumny numerów wierszy. Poniższy kod wykonaj w głównym projekcie i sprawdź pliki w panelu Files; nazwę katalogu zachowaj dla Z02.

```
dir.create("zadania/z02/wyniki", recursive = TRUE, showWarnings = FALSE)
utils::write.csv(dane, "zadania/z02/wyniki/wyczyszczone.csv",
  row.names = FALSE, fileEncoding = "UTF-8", na = "NA")
utils::write.csv(dziennik, "zadania/z02/wyniki/dziennik.csv",
  row.names = FALSE, fileEncoding = "UTF-8")
```

Z02 — własny wariant ankiety

W `zadania/z02/analiza.R` odczytaj własny `dane/surowe.csv`, sprawdź N, nazwy, polskie etykiety oraz klasy. Usuń wyłącznie identyczne kopie całych wierszy, zamień kod 99 w pozycji 2 na NA i oznacz jako brak czasu poza 0–120. Zapisz liczbę każdej zmiany, N przed/po i liczby braków; nie usuwaj wszystkich niekompletnych `id_odpowiedziow`. Zapisz `wyczyszczone.csv` i `dziennik.csv` w katalogu wyników.

W `odpowiedzi.md` opisz reguły, liczby zmian i różnicę między zerem a brakiem. Wyjaśnij klasę i skalę grupy, czasu oraz pojedynczej pozycji. Dodaj odnośniki do obu zapisanych CSV i polecenie uruchomienia skryptu od początku. Pracujesz na własnym ID i scenariuszu; wartości nie muszą być takie same jak we wspólnej demonstracji.

Kryterium	Punkty
Z02.1 Import, kodowanie, N i struktura	2
Z02.2 Typy, kody braków i duplikaty	4
Z02.3 Jawny dziennik, N i brak/zero	2
Z02.4 Odtwarzalny skrypt i eksport CSV	2

Pełne poziomy punktacji: `rubryka("Z02")`. Kontrola techniczna nie potwierdza, że wszystkie decyzje o czyszczeniu są uzasadnione.

Oddanie i pytania kontrolne

Zapisz pliki i uruchom skrypt od początku w głównym projekcie. Kontrola musi odtworzyć wyniki z surowego pliku; ręczna poprawka arkusza poza skryptem nie wystarcza. Po wysłce sprawdź pokwitowanie GitHub, a przed opuszczeniem sali wyloguj również przeglądarkę.

```
sprawdz_zadanie("Z02")
oddaj_zadanie("Z02")
status_oddania("Z02")
wyloguj_github()
```

Dlaczego 99 nie jest oceną? Dlaczego usunięcie wiersza za jeden brak traci inne odpowiedzi? Jak odróżnisz dwie identyczne kopie od sprzecznych odpowiedzi o tym samym ID? Co sprawdzisz, gdy CSV wczyta się jako jedna kolumna? Odpowiedz, odwołując się do własnego dziennika.